Command Cloud to Dividia: integration reference

Everything your backend needs to send a ticket to Dividia and get its PDF back. You do not need to run the tester (`server.js`) to use this; the tester is the same two requests, working. **Any language:** these are plain HTTPS requests. Node.js is needed only to run the tester, not for what you build.

1. **Dividia gives Command two values for each customer:** a Customer ID and a customer secret, when the customer is set up in ScaleWatcher Cloud. The customer does not get them.
2. **You store them on your servers** before that customer's first ticket.
3. **Your servers make every call to Dividia.** No browser and no customer computer calls Dividia.

1. The values

Value	Sent to Dividia as	Comes from	What it does
Customer ID	**header** `Dividia-Customer-Id`	your record of the customer	Says which customer the request is for.
Customer secret	**header** `Authorization: Bearer`	your record of the customer	Proves the request is from Command. Send the whole value, including `whsec_`.
Site id	**body** `location.id`	the ticket	Picks the site at Dividia.
Site name	**body** `location.name`	the ticket	For display. Does not route.
Scale id	**body** `scaleId`	the ticket	Picks the scale at that site.

The secret does not expire. Dividia can replace it (the old one works for 24 more hours) or stop it immediately.

2. Send a ticket

```
POST https://api.cloud.dividia.net/sw/webhooks/cc
Authorization: Bearer <customer secret>
Dividia-Customer-Id: <Customer ID>
Content-Type: application/json

{
  "uniqueId": "8f1c0f0a-5b0e-4a57-9a55-0c1d2e3f4a5b",   required, new for every ticket
  "ticketNumber": "70421",                              required
  "ticketDateTime": "2026-11-11T15:18:42Z",
  "vehicle": {
    "id": "VEH-7908", "name": "Truck 218", "autoId": "AUTO-218",
    "gross": { "value": 74250, "uom": "lb" }, "tare": { "value": 27100, "uom": "lb" },
    "net": { "value": 47150, "uom": "lb" }, "maxGVW": { "value": 80000, "uom": "lb" }
  },
  "carrier":  { "id": "CAR-1", "name": "Example Hauling" },
  "customer": { "id": "CUS-1", "name": "Example Customer" },
  "project":  { "id": "PRJ-1", "name": "Example Project" },
  "product":  { "id": "PRD-766", "name": "Fill dirt",
                "quantity": { "value": 47150, "uom": "lb" } },
  "orderId": "ORD-1",
  "location": { "id": "<location.id>", "name": "<location.name>" },   picks the site
```

```
  "scaleId": "<scaleId>"                                    picks the scale
}
```

Only `uniqueId` and `ticketNumber` are required. The same `uniqueId` never creates a second ticket, so sending a ticket again is always safe.

Dividia's answer, and what to do

Answer	Meaning	What to do
`200 {"status":"delivered"}`	Dividia has the ticket and sent it to the site.	Done.
`200 {"status":"fallback"}`	Dividia has the ticket. The site or scale was not reached just now; `reason` says why.	Done.
`200 {"status":"duplicate"}`	Dividia already had this `uniqueId`.	Done.
`200 {"status":"failed"}`	The `location.id` is not set up at Dividia.	Do not resend. Check the `location.id`.
`400`	The body is wrong. The message names the field.	Fix it, then send.
`401`	Wrong Customer ID or secret.	Stop and check both values.
`429`, `500`, `503`, no answer	Temporary.	Send the same ticket again, with the same `uniqueId`. Keep it in a queue until it is accepted.

---

3. Get a ticket's PDF

```
GET https://api.cloud.dividia.net/sw/api/tickets/pdf
    ?locationId=<location.id>
    &ticket=<ticket number>
Authorization: Bearer <customer secret>
Dividia-Customer-Id: <Customer ID>
```

The same two headers. `locationId` is the ticket's `location.id`.

Dividia's answer, and what to do

Answer	Meaning	What to do
`200 application/pdf`	The PDF. It has no photos until the site has recorded the weighment.	Use it. Ask again later for the complete PDF.
`200 {"matches":0}`	Dividia has no such ticket at that location.	Check the ticket number and `location.id`.
`200 application/zip`	The ticket number is on more than one transaction there.	The ZIP holds each PDF.
`400`	The request is wrong. The message says what.	Fix it.

`401`	Wrong Customer ID or secret.	Check both values.
`403`	The location is not this customer's.	Check the `location.id`.
`429`	Too many requests.	Wait for the time in the `Retry-After` header, then ask again.
`500`, no answer	Temporary.	Ask again.

File name. The `Content-Disposition` header names the file: `<Dividia site number>-<location.id>-ticket_<ticket number>.pdf`, for example `1234-PLANT-7-ticket_70421.pdf` (characters other than letters, digits and `-` become `-`). Keep it or use your own.

4. Limits and network

Item	Value
Sending tickets	6,000 per 15 minutes from one server address.
Getting PDFs	60 per minute for one customer, one at a time for that customer.
All other requests	1,000 per 15 minutes from one server address, across all customers. PDF requests count toward this; ticket sends do not.
Network	Outbound HTTPS (port 443) from your servers to `api.cloud.dividia.net`.
CORS	Does not apply: your server calls Dividia, not a browser. A call made straight from a web page is blocked by the browser, by design.